Getting Started with AI Tools

AI Bootcamp for UVM Economics - Session 1

Emily Beam Erkmen Aslim

2026-04-22

Welcome

What is this?

Two lunch sessions on AI tools for teaching and research.

- **Today**: getting started, what the tools do, live demos
- **April 27**: research workflows, web scraping, automated grading
- no installation required today
- watch now, ask questions, try things later

We are still learning

This is a new space, for us and for the tools.

- we started using these tools in December and January while working on the Economics AI letter
- the learning curve is real, but it is short
- things are changing quickly
- some of what feels hard now may get much easier very soon

The AI ladder

Where are you today?

Level	Description
0	Haven’t tried it
1	Tried it once, wasn’t impressed
2	Use occasionally for simple tasks
3	Regular user, integrated into workflow
4	Power user, multi-step workflows
5	Building systems with AI

Adapted from Kyle Jensen’s AI adoption framework, as described in Goldsmith-Pinkham (2026).

Goal today: show you what Levels 3-4 look like.

How the tools differ

Conversational AI vs Agentic AI

The useful distinction is not the brand name. It is what the tool can actually do for you and how directly it can work on your project.

Conversational AI	Agentic AI
best for question-answering, drafting, summarizing, and quick help	best for multi-step work on real projects
usually one conversation at a time	can support multiple simultaneous workflows
you can upload files and sometimes do light coding tasks	can work across as many or as few files as you allow

Conversational AI	Agentic AI
tends to be session-by-session	can be shaped into more customized systems
easier to start with	more setup, but often more quality and efficiency

In practice, we use both. We often run multiple Claude and Codex sessions at the same time with different tasks separated into different windows.

Vocabulary

A few shared terms will make the rest of the session easier to follow.

Term	What we mean
Agent	A tool instance carrying out a bounded task with some autonomy
Model	The underlying AI system you are talking to
Session	One conversation or one working thread with the tool
Context window	The working memory available in the current session
Artifact	A concrete output: file, draft, memo, script, table, or slide
Rule file	A standing instructions file such as `CLAUDE.md` or `AGENTS.md`
Project file	A project-specific instructions file that lives with the project itself
Permissions	The rules about what the tool may read, change, run, or send
Orchestration	Coordinating several tasks, tools, or agents in one workflow

The tools landscape

Conversational AI

- ChatGPT
- Claude.ai
- good for quick questions, drafting, summarizing, and uploaded files
- lower barrier to entry

Agentic AI

- Claude Code
- Codex
- good for file access, iteration, reusable workflows, and longer jobs
- usually the better option when you want real outputs instead of just advice

> UVM provides enterprise-level Microsoft Copilot with higher data security and strong Microsoft Office integration, but it does not support the agentic workflows we are focusing on here.

Why file access matters

The jump from conversational AI to agentic AI is mostly about direct work on real files.

- less copy-paste
- easier iteration across many files
- persistent outputs that stay in your project
- easier to build workflows you can reuse later
- much better fit for real research, teaching, and website work

Working with Projects

About `CLAUDE.md` and `AGENTS.md`

For coding tools, these are standing instruction files that are loaded automatically when you start working in a project that contains them.

- they store stable preferences, permissions, workflow rules, and project context
- they reduce how much you need to re-explain every session
- you can have both general and project-specific versions
- they should stay short: if they get too long, they slow things down and important rules get lost

Personal preferences example

Insert screenshot of short preferences file or EB-VOICE example

- good place for preferences like full file paths, permission rules, and writing defaults
- a file like `EB-VOICE.md` is one concrete example
- after a work session, you can update the file based on your edits
- keep it short enough that it stays useful

Markdown

Markdown (`.md`)

AI tools often write in Markdown because it is a simple plain-text format that converts easily into other outputs.

- common output for notes, reports, instructions, and drafts
- easy to convert into `docx`, `pdf`, `html`, slides, and other formats
- useful to get comfortable recognizing it, even if you do not write it yourself
- recommended viewers/editors: Typora ($15), MarkText, Zettlr, MacDown

Raw Markdown vs rendered Markdown

Markdown looks ugly in raw form and normal in a viewer.

Raw Markdown

getting-started-terminal-tools.md

```
# Getting Started: Ghostty, Zellij, and Oh My Zsh

A quick guide to what each tool does, how they layer together, and the minimum you need to get productive.

---

## How these three tools fit together
```
┌──┐
│ Ghostty   — the terminal app   │
│ ┌──────────────────────────────────────┐│
│ │ Zellij   — splits & sessions   │
│ │ ┌──────────────┐ ┌────────────────┐│
│ │ │ Pane 1   │ │ Pane 2   │
│ │ │ (zsh + OMZ) │ │ (zsh + OMZ)   │
│ │ └──────────────┘ └────────────────┘│
│ └──────────────────────────────────────┘│
└──┘
```

- **Ghostty** replaces Terminal.app or iTerm2. It's the window you open.
- **Zellij** runs *inside* Ghostty. It gives you split panes, tabs, and persistent sessions.
- **Oh My Zsh** runs inside each shell. It makes your prompt prettier and adds shortcuts.

They don't conflict — each operates at a different layer.

---

## 1. Ghostty (terminal emulator)

### What it is
A fast, GPU-accelerated terminal app for macOS. Replaces Terminal.app or iTerm2.

### How to use it
- **Open it**: Launch from `/Applications/Ghostty.app` (Spotlight: type "Ghostty")
- **Set as default**: System Settings → Desktop & Dock → Default terminal application → Ghostty
- That's it — you just use it like you used Terminal.app. Everything else (zsh, commands, etc.) works the same
inside it.

### Key shortcuts
| Action | Shortcut |
|---|---|
| New window | `Cmd+N` |
| New tab | `Cmd+T` |
| Split right | `Cmd+D` |
| Split down | `Cmd+Shift+D` |
```

Rendered preview

Getting Started: Ghostty, Zellij, and Oh My Zsh

A quick guide to what each tool does, how they layer together, and the minimum you need to get productive.

How these three tools fit together

```
┌──────────────────────────────────────────┐
│  Ghostty          — the terminal app     │
│  ┌──────────────────────────────────────┐│
│  │  Zellij        — splits & sessions   │
│  │  ┌──────────────┐  ┌────────────────┐│
│  │  │ Pane 1       │  │ Pane 2         │
│  │  │ (zsh + OMZ)  │  │ (zsh + OMZ)    │
│  │  └──────────────┘  └────────────────┘│
│  └──────────────────────────────────────┘│
└──────────────────────────────────────────┘
```

- **Ghostty** replaces Terminal.app or iTerm2. It's the window you open.
- **Zellij** runs *inside* Ghostty. It gives you split panes, tabs, and persistent sessions.
- **Oh My Zsh** runs inside each shell. It makes your prompt prettier and adds shortcuts.

They don't conflict — each operates at a different layer.

1. Ghostty (terminal emulator)

What it is

A fast, GPU-accelerated terminal app for macOS. Replaces Terminal.app or iTerm2.

How to use it

- **Open it**: Launch from `/Applications/Ghostty.app` (Spotlight: type "Ghostty")
- **Set as default**: System Settings → Desktop & Dock → Default terminal application → Ghostty
- That's it — you just use it like you used Terminal.app. Everything else (zsh, commands, etc.) works the same inside it.

Working Memory

Context window

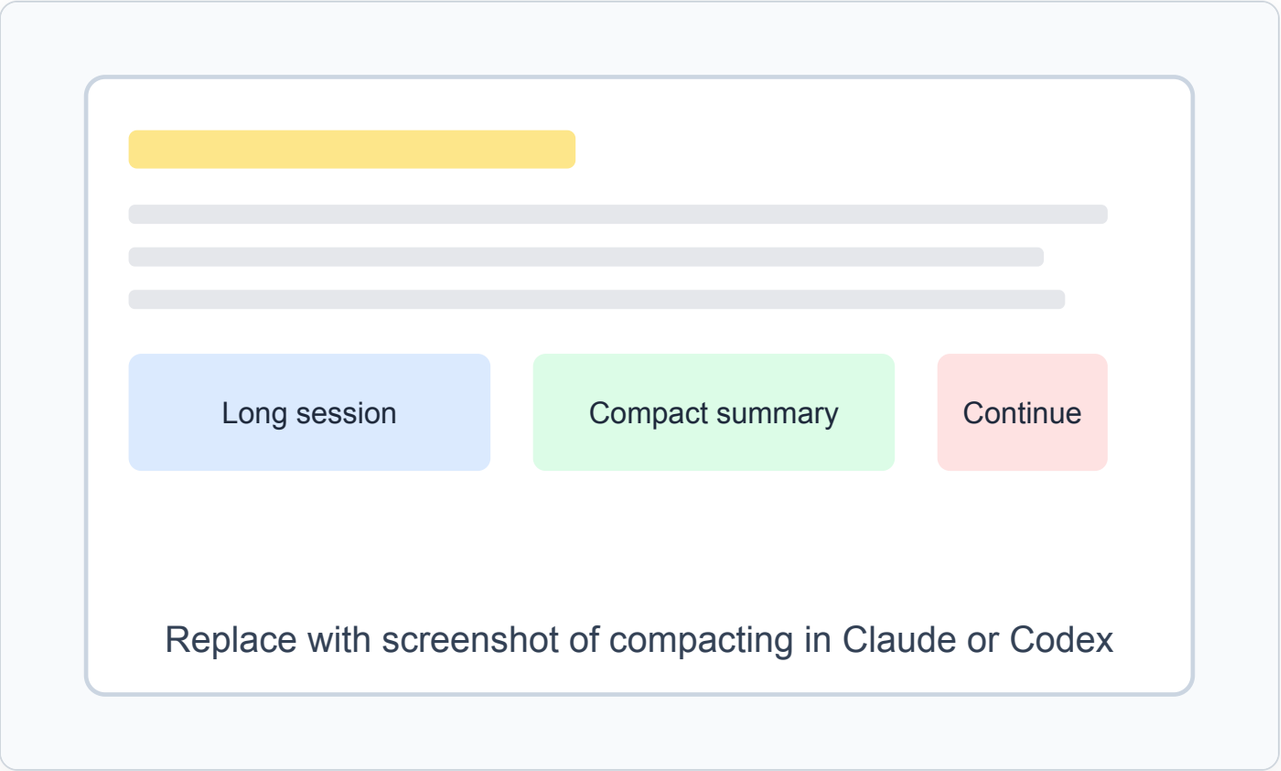
The context window is the tool’s working memory for the current session.

- every model has a maximum context window set by the model or the company
- as that working memory fills up, performance can get worse
- larger, messier sessions are more likely to accumulate mistakes
- this is one reason to separate tasks and work in multiple sessions

Workflow schematic

For larger tasks, it helps to separate planning, execution, and review rather than doing everything in one long messy thread.

Compacting



- compacting summarizes the useful state, then lets you continue with a cleaner session
- it is most helpful for larger tasks, not every tiny edit
- Codex is currently better at doing this automatically
- Claude is improving, but it is still useful to think about this explicitly

Safety

Safety example

Sensitive work can often be redesigned instead of abandoned.

- do not give it PII-heavy files unless the rules and platform are appropriate
- let it work on code and log files instead of raw protected data
- if a task is too risky, do not do it
- paid subscriptions generally let you turn off training on your content, and that is recommended

What breaking your computer really means

This risk is usually low unless you deliberately ask for something extreme.

- the tool can make mistakes
- the tool may pause and ask for permission on risky actions
- you can refuse, stop the task, or tighten the rules

Collaboration

GitHub and collaboration

These tools work especially well with GitHub-based workflows, though GitHub is not required.

- very useful for collaboration and version control
- especially powerful for intensive or shared coding projects
- makes it easier to track changes, compare versions, and work with others
- a power-up, not a prerequisite

What to expect from the demos

You are about to see practical examples, not magic.

- live prompting
- iteration
- occasional waiting
- reviewing output critically

Wrap-up

The biggest jump in usefulness comes when the AI can work directly with your files.

Next session: research pipelines, web scraping, automated grading.